

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Mohammad Fuzail / 192565050 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 – Hospital Shift Scheduling (Backtracking Search)

Assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night). Constraints: (i) D1 cannot work Night; (ii) D2 must work before D3 (Morning < Afternoon < Night); (iii) only one doctor per shift; (iv) D3 cannot work Morning; (v) every doctor gets exactly one shift. Apply Backtracking Search with all intermediate steps and constraint checks; state the final schedule.

CSP formulation

- **Variables:** D1, D2, D3 (the shift of each doctor). **Domain:** {Morning (M), Afternoon (A), Night (N)} with $M < A < N$.
- **Constraints:** C1: $D1 \neq N$; C2: $D2 < D3$ (in time); C3: all-different (one doctor per shift, and with 3 doctors / 3 shifts every doctor gets exactly one – C5); C4: $D3 \neq M$.
- **Search order:** variables $D1 \rightarrow D2 \rightarrow D3$, values tried in order M, A, N. After each assignment the partial assignment is checked against all constraints that involve only assigned variables; on a violation the algorithm backtracks.

Step	Partial assignment tried	Constraint check	Result
1	D1 = M	C1: $M \neq N$ ✓	Consistent → go deeper
2	D1 = M, D2 = M	C3: M already used by D1 ✗	Violated → try next value
3	D1 = M, D2 = A	C3 ✓ (M, A different); C1 ✓	Consistent → go deeper
4	D1 = M, D2 = A, D3 = M	C4: $D3 \neq M$ ✗ (also C3: M used)	Violated → try next value
5	D1 = M, D2 = A, D3 = A	C3: A already used by D2 ✗ (also C2: $A < A$ false)	Violated → try next value
6	D1 = M, D2 = A, D3 = N	C1 ✓ (D1 = M); C2: $A < N$ ✓; C3 ✓ all different; C4: $N \neq M$ ✓; C5 ✓ all 3 assigned	All constraints satisfied → SOLUTION

Search tree (DFS with backtracking):

```

D1 = M (ok)
+-- D2 = M X (all-different violated) <- backtrack
+-- D2 = A (ok)
|   +-- D3 = M X (D3 cannot be Morning) <- backtrack
|   +-- D3 = A X (shift already used) <- backtrack
|   +-- D3 = N OK -> all constraints satisfied => SOLUTION

```

Final valid schedule:

Shift	Morning	Afternoon	Night
Doctor	D1	D2	D3

Note: if the search is continued after the first solution, the only other valid schedule is D2 = Morning, D1 = Afternoon, D3 = Night ($D1 \neq \text{Night}$, D2 before D3, $D3 \neq \text{Morning}$). The first solution found by backtracking is D1–Morning, D2–Afternoon, D3–Night.

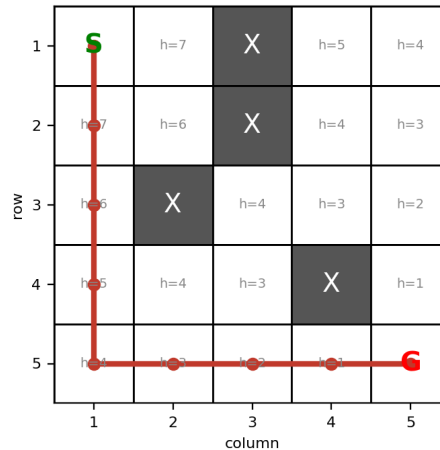
Question 2 – Robot in a 5×5 Grid (BFS and Manhattan-distance priority queue)

A robot moves from Start (S) to Goal (G) in a 5×5 grid with obstacles. Moves: Up, Down, Left, Right; cost 1 per move; cannot pass through obstacles; Manhattan distance heuristic. Using BFS, show step-by-step node expansion and priority-queue updates, and determine the optimal path and cost.

Problem set-up (the grid figure is not given in the paper, so the following grid is assumed)

Cells are written (row, column). Start S = (1,1), Goal G = (5,5). Obstacles (X) = (1,3), (2,3), (3,2), (4,4).

Neighbours are generated in the order Up, Down, Left, Right. Heuristic $h(n) = |r - 5| + |c - 5|$ (Manhattan distance to G); each cell shows its h value.



A. Breadth-First Search (FIFO queue – expands level by level)

Step	Expanded node	Children added to queue (parent)	Queue (front → back) after step
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(1,2), (3,1), (2,2)
3	(1,2)	–	(3,1), (2,2)
4	(3,1)	(4,1)	(2,2), (4,1)
5	(2,2)	–	(4,1)
6	(4,1)	(5,1), (4,2)	(5,1), (4,2)
7	(5,1)	(5,2)	(4,2), (5,2)
8	(4,2)	(4,3)	(5,2), (4,3)
9	(5,2)	(5,3)	(4,3), (5,3)
10	(4,3)	(3,3)	(5,3), (3,3)
11	(5,3)	(5,4)	(3,3), (5,4)
12	(3,3)	(3,4)	(5,4), (3,4)
13	(5,4)	(5,5)	(3,4), (5,5)

BFS stops when G is generated. Path found by following parents: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5). Cost = 8. BFS is optimal here because every move costs 1, but it explores nodes in all directions.

B. Priority queue with Manhattan heuristic, $f(n) = g(n) + h(n)$

Step	Expanded (g, h, f)	Priority queue after expansion – node : f (lowest first)
1	(1,1) g=0, h=8, f=8	(2,1):8, (1,2):8
2	(2,1) g=1, h=7, f=8	(3,1):8, (2,2):8, (1,2):8
3	(3,1) g=2, h=6, f=8	(4,1):8, (2,2):8, (1,2):8
4	(4,1) g=3, h=5, f=8	(5,1):8, (4,2):8, (2,2):8, (1,2):8
5	(5,1) g=4, h=4, f=8	(5,2):8, (4,2):8, (2,2):8, (1,2):8

Step	Expanded (g, h, f)	Priority queue after expansion – node : f (lowest first)
6	(5,2) g=5, h=3, f=8	(5,3):8, (4,2):8, (2,2):8, (1,2):8
7	(5,3) g=6, h=2, f=8	(5,4):8, (4,2):8, (2,2):8, (1,2):8, (4,3):10
8	(5,4) g=7, h=1, f=8	(5,5):8, (4,2):8, (2,2):8, (1,2):8, (4,3):10
9	(5,5) g=8, h=0, f=8	GOAL reached – stop

Optimal path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5). **Optimal cost = 8** (equal to the Manhattan distance 8, so no shorter path exists).

Moves: Down, Down, Down, Down, Right, Right, Right, Right. The priority-queue search expanded fewer nodes (9) than plain BFS (13) because the heuristic guides it toward G; both give the same optimal cost 8.

Question 3 – Autonomous Rescue Robot (Online Search under Partial Observability)

A rescue robot in a partially observable building (grid of rooms) must go from S to the survivor at G. It moves Up/Down/Left/Right, some cells are blocked, it senses only adjacent cells, a move costs 1 plus an extra 2 when entering a risky zone, it must minimise total cost safely and update its knowledge online. Tasks: (a) analyse the constraints, (b) develop the solution approach, (c) demonstrate AI concepts, (d) justify.

a) Analysis of constraints and their effect on decisions

Constraint	Effect on the robot's decision-making
4-direction moves	Branching factor ≤ 4 ; no diagonal shortcuts, so paths are Manhattan-like and the heuristic h = Manhattan distance is admissible.
Blocked cells	Actions into obstacles are illegal; the plan must route around them and may need re-planning when a new block is discovered.
Sensing only adjacent cells (partial observability)	The full map is unknown, so the robot cannot compute the whole path in advance. It must keep an internal map (belief state), act, sense, and update.
Cost 1 per move, +2 for a risky cell (cost 3)	Not all paths of equal length are equal: a longer path through safe cells may be cheaper than a short path through risky cells (one risky cell = 2 extra steps). Cost-sensitive search is needed.
Minimise cost with safe navigation	Two objectives: low cost and safety. Risky cells are allowed only when the detour needed to avoid them costs more than 2 per risky cell; safety is enforced by the cost penalty (and a hard ban on blocked cells).
Online / dynamic knowledge update	Planning and execution are interleaved: after every move the robot updates the map and re-plans if the current plan is no longer valid.

b) Solution approach: Online search agent using Uniform Cost Search (UCS / A*) with re-planning

- **Step 1 – Initialise:** internal map $M = \{S \text{ known, everything else unknown}\}$; current cell = S; goal = G.
- **Step 2 – Sense:** observe the four adjacent cells and record each as free, blocked or risky (cost 1 / ∞ / 3).
- **Step 3 – Plan:** run UCS (or A* with admissible h = Manhattan distance) on the *known* map, treating unknown cells as free with cost 1 (optimistic assumption). The priority queue is ordered by $g(n)$ = sum of step costs.
- **Step 4 – Act:** execute only the *first* move of the cheapest path.
- **Step 5 – Update:** sense again from the new cell and update M (new obstacles, new risky zones).
- **Step 6 – Re-plan:** if the remaining path now crosses a blocked cell or a cheaper path exists, run UCS again from the current cell. Repeat Steps 2–6 until the robot reaches G (survivor found), or no path exists (report failure / request help).

```
function ONLINE-RESCUE-AGENT(percept) returns an action
  persistent: map (known cells with types), pos, goal G, plan
  UPDATE-MAP(map, pos, percept)           // adjacent cells: free / blocked / risky
  if pos = G then return STOP              // survivor reached
  if plan is empty or PLAN-BLOCKED(plan, map) then
    plan <- UCS(map, pos, G)               // step cost = 1, or 3 if risky;
                                           // unknown cells assumed free (cost 1)
    if plan = failure then return REQUEST-HELP
  action <- FIRST(plan) ; plan <- REST(plan)
  pos <- RESULT(pos, action)
  return action
```

c) Application of AI concepts

- **Intelligent agent:** the robot is a PEAS agent. *Performance:* survivor reached, minimum total cost, safety (no damage). *Environment:* disaster building. *Actuators:* wheels / motors (up, down, left, right). *Sensors:* adjacent-cell sensors, cameras, position sensor. It is a **model-based, goal-based agent with cost (utility) awareness** – it keeps an internal map, has a goal, and compares paths by cost.
- **Rationality:** at each step the robot chooses the action expected to maximise the performance measure (minimum total cost, safely) given what it has perceived so far and its stored knowledge. Re-planning when new percepts arrive, and accepting a risky cell only when the safe detour costs more than 2 extra, is rational behaviour.
- **Environment type:** partially observable (only adjacent cells), single agent, deterministic (a move succeeds as intended), sequential (earlier moves affect later cost), dynamic / unknown (map is discovered and blocks may appear), discrete (grid cells), and unknown (an exploration / online problem).

d) Why this approach is most suitable

- **Handles partial observability:** offline search (BFS, A* over a full map) needs complete knowledge; an online agent plans with what it knows and corrects itself as it senses more.
- **Handles unequal step costs:** BFS and DFS ignore costs and would treat a risky cell like a safe one; UCS expands by lowest cumulative cost, so it is optimal for costs 1 and 3, and finds the cheapest safe route.
- **Safe and complete:** UCS is complete and optimal for non-negative costs on the known map; re-planning guarantees that new obstacles never cause the robot to follow an invalid plan.
- **Efficient:** adding the Manhattan heuristic (A*) keeps the cost-optimal behaviour but reduces the number of expanded cells; only one move is executed per plan, so the robot reacts quickly to new information.
- **Conclusion:** an online search agent using UCS / A* with continuous re-planning best satisfies all constraints – limited sensing, blocked cells, risk-weighted costs and dynamic knowledge updates.